WEEK 2 · DAY 1 · OVERVIEW

Logistic Regression and Odds



Created by **Jeff Lenamon**

ABOUT 55 MIN WITH EXERCISES / 8 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Week 1 asked how wide a spread is; credit work also asks whether a borrower pays.

A public file of 30,000 credit card accounts records whether each one defaulted the following month.

The accounts are real, in Taiwan in 2005, so nothing here carries over to another market or year.

SEX, **EDUCATION**, **MARRIAGE** and **AGE** are never features: a model using them would sort people by them, and leaving them out does not make a model neutral, because other columns can carry the same information.

The output is a model probability of default in this file, never a credit decision about anyone.

OBJECTIVES

Fit a logistic regression by eye and then by computer, read its coefficients as odds ratios with their units, and score its probabilities on accounts it never saw.

ABOUT 55 MIN WITH EXERCISES / 8 SECTIONS / 4 EXERCISES

Your Route Through Day 1

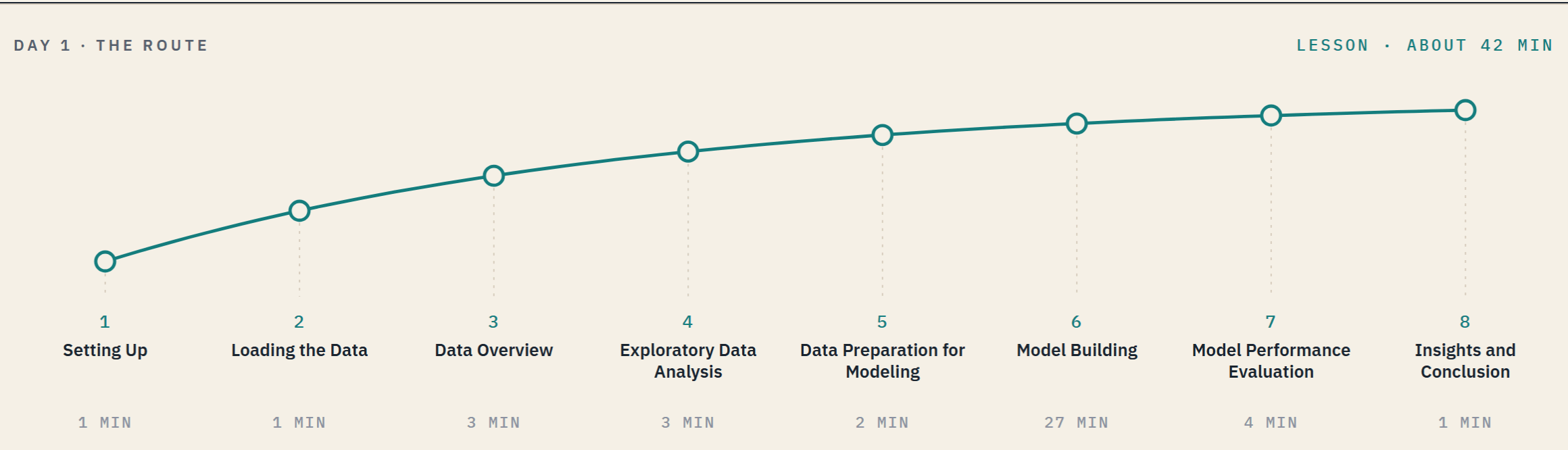


FIGURE 1 Eight sections, in order, with about how long each takes.

Time. About 42 minutes for the lesson, then about 12 for the four exercises.

Setting Up

THE SEED

```
import os
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
```

THE DATA FOLDER

```
from pathlib import Path
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
```

THE LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from ipywidgets import interact
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
import statsmodels.api as sm
from sklearn.metrics import log_loss
```

Run this cell and move on: it fixes the seed and finds the course data.

Loading the Data

READ THE FILE

```
# the 30,000 card accounts
card = pd.read_csv(DATA + "credit_card_default.csv")
# the target's name has spaces, so we keep it in a variable
TARGET = "default payment next month"
```

THE DATA DICTIONARY

- **ID**: a row label, never a feature
- **LIMIT_BAL**: the credit limit, NT dollars
- **PAY_0**, **PAY_2** to **PAY_6**: repayment status, September back to April (-1 paid duly, 1 to 9 months late, -2 and 0 undocumented)
- **BILL_AMT1** to **BILL_AMT6**: the bill, NT dollars, September back to April
- **PAY_AMT1** to **PAY_AMT6**: the amount paid, NT dollars, September back to April
- **SEX**, **EDUCATION**, **MARRIAGE**, **AGE**: about the person, never features
- **default payment next month**: 1 if the account defaulted the following month, 0 if not, **the target**

Data Overview

THE FIRST ROWS

	ID	LIMIT_BAL	PAY_0	PAY_AMT1	DEFAULT PAYMENT NEXT MONTH
0	1	20000	2	0	1
1	2	120000	-1	0	1
2	3	90000	0	1518	0
3	4	50000	0	2000	0

SHAPE

```
card.shape
```

```
(30000, 25)
```

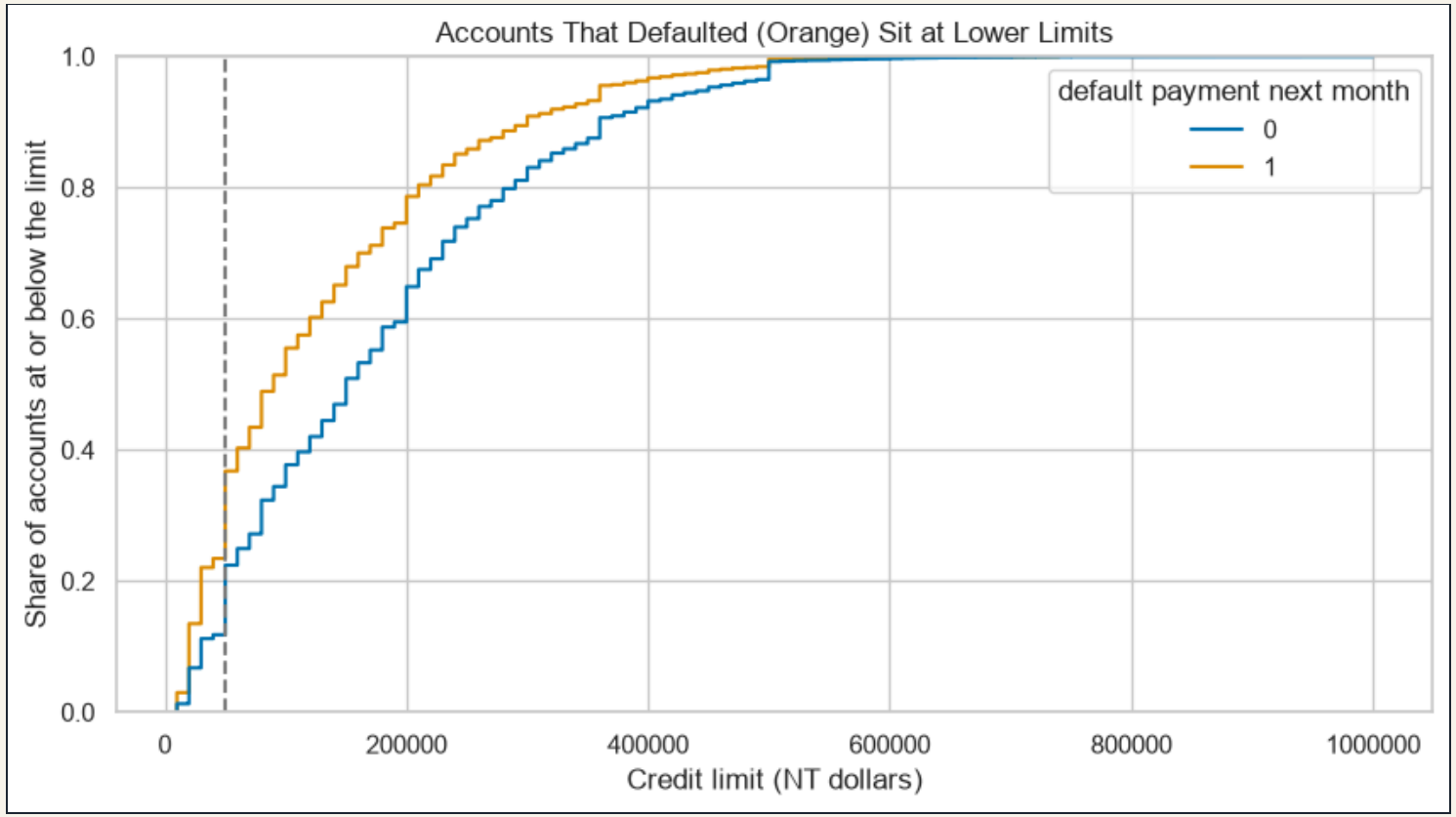
COMPLETE?

```
print("Missing values:", card.isnull().sum().sum())
```

```
Missing values: 0
```

The file is complete; the only decision is which columns may be features, and the Business Context made it.

Exploratory Data Analysis



MEDIAN AND THE SHARE AT 50,000

```
default payment next month
0   150,000.0000
1    90,000.0000
Name: LIMIT_BAL, dtype: float64
default payment next month
0   0.2241
1   0.3677
Name: LIMIT_BAL, dtype: float64
```

Lower limits go with more defaults. Next, the week's split; repayment status waits for the training accounts.

Data Preparation for Modeling

The week's split: 30,000 accounts, cut 60, 20 and 20 percent, the default rate kept in each part

18,000	6,000	6,000
training accounts	validation accounts	test accounts
3,982 defaulted	1,327 defaulted	1,327 defaulted
22.12%	22.12%	22.12%
fit every model, Days 6 to 9	check each model, Days 6 to 9	closed until Day 10, opened once

stratify=y keeps the default rate the same in every part; random_state=SEED keeps the same accounts in every part on every run.

FIGURE 2 The 30,000 accounts cut into 18,000 training, 6,000 validation and 6,000 test, each defaulting at 22.12 percent.

SIZES AND DEFAULT RATES

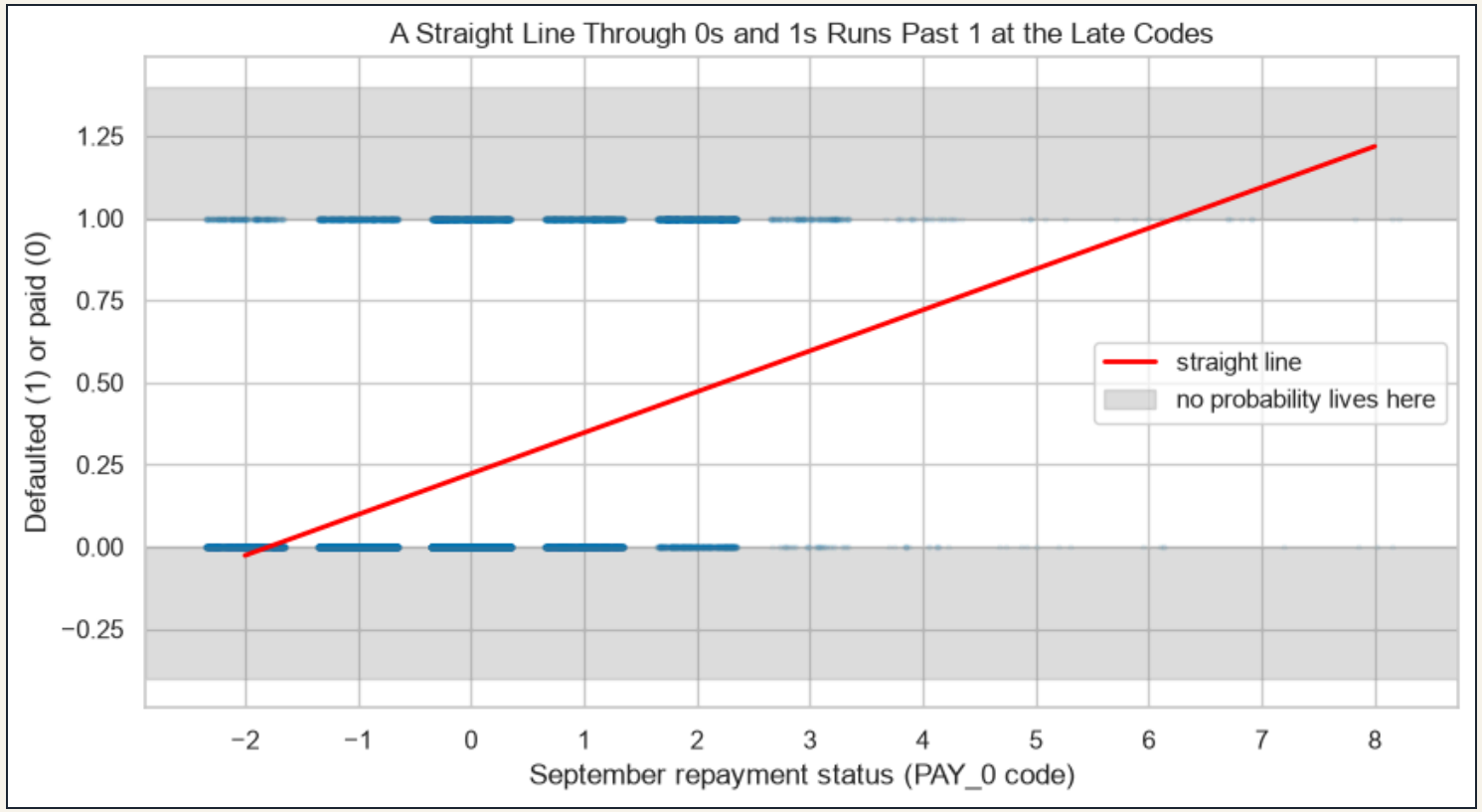
```
Training: 18000  Validation: 6000  Test: 6000
Default rate in each: 0.2212 0.2212 0.2212
```

THE TWO CUTS

```
X_rest, X_test, y_rest, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEED)
X_train, X_valid, y_train, y_valid = train_test_split(X_rest, y_rest, test_size=0.25, stratify=y_rest, random_state=SEED)
```

Days 6 to 9 fit on training accounts and check on validation accounts; the test accounts stay closed until Day 10.

Model Building



THE LINE'S FIT

```
default = ( 0.1245 ) * PAY_0 + ( 0.2226 )
```

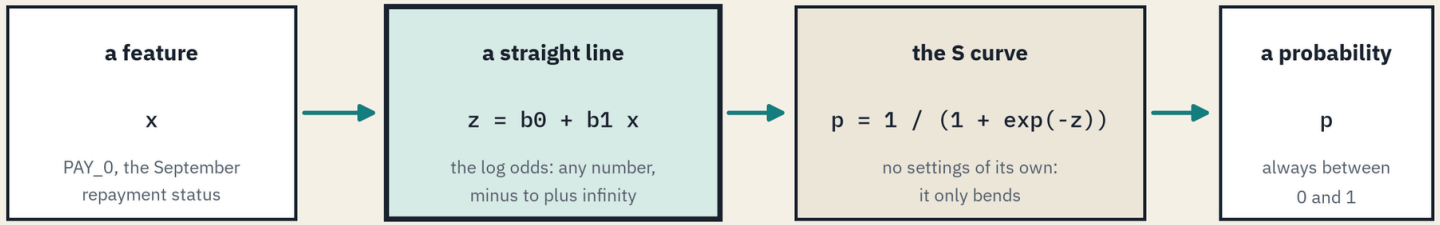
ON VALIDATION ACCOUNTS

```
Lowest: -0.0264  Highest: 1.2185
Below 0: 578  Above 1: 12
```

A probability lives between 0 and 1, so a straight line needs a scale that can take any value. Log odds are that scale.

From a Line to the S Curve

A logistic regression: a straight line inside an S curve



b0 and b1 are the only two numbers the fit chooses

Read backwards: a probability turned into log odds, ln(p / (1 - p)), is a straight line in x.

FIGURE 3 A feature feeds a straight line, the log odds, which the S curve bends into a probability between 0 and 1.

THE COEFFICIENTS

	COEF	STD ERR	P-VALUE
const	-1.4185	0.0203	0.0000
PAY_0	0.7702	0.0187	0.0000

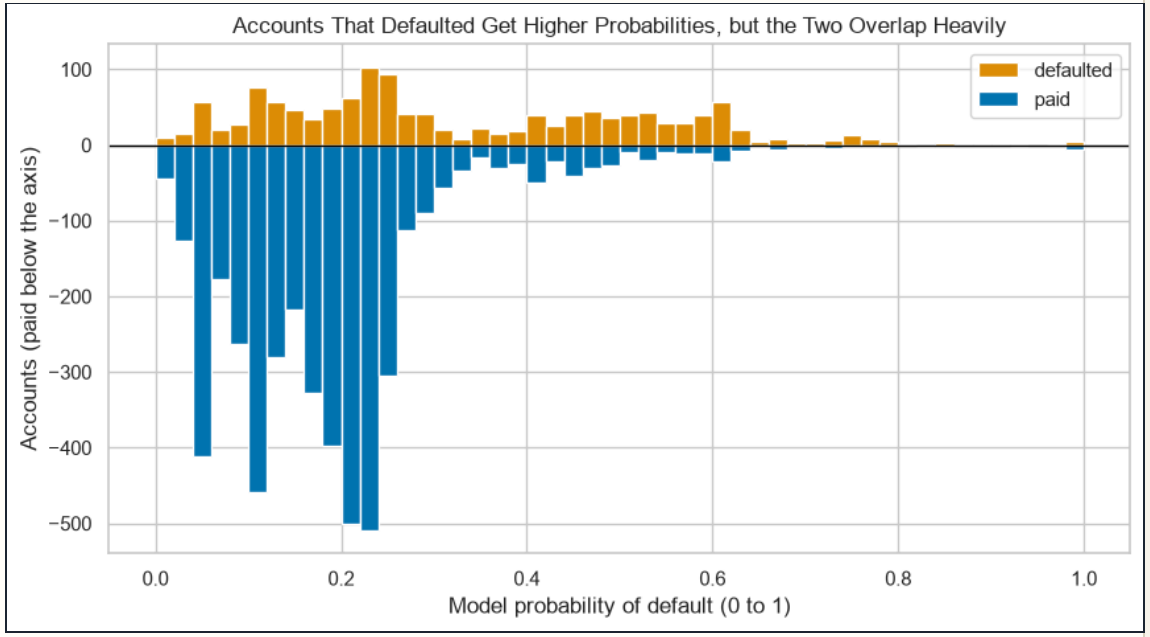
READ AS AN ODDS RATIO

```
Odds ratio per code step: 2.1602
Model probability at code 0: 0.1949
```

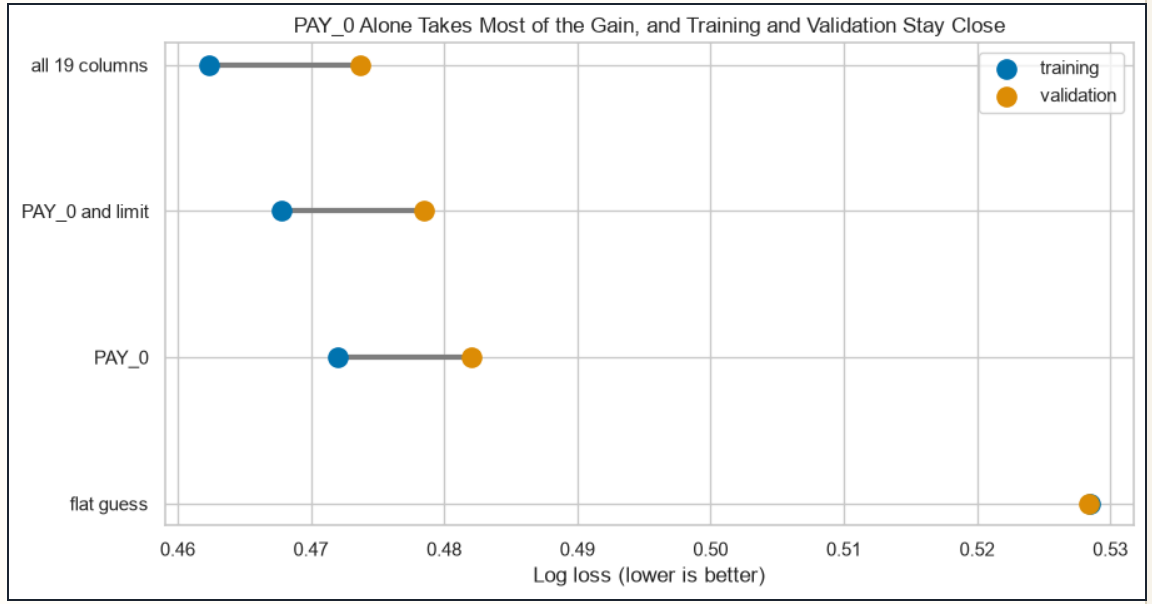
An odds ratio, e to the power of a coefficient, says how many times as high the odds are after one step of the feature.

Model Performance Evaluation

DEFAULTED ABOVE, PAID BELOW

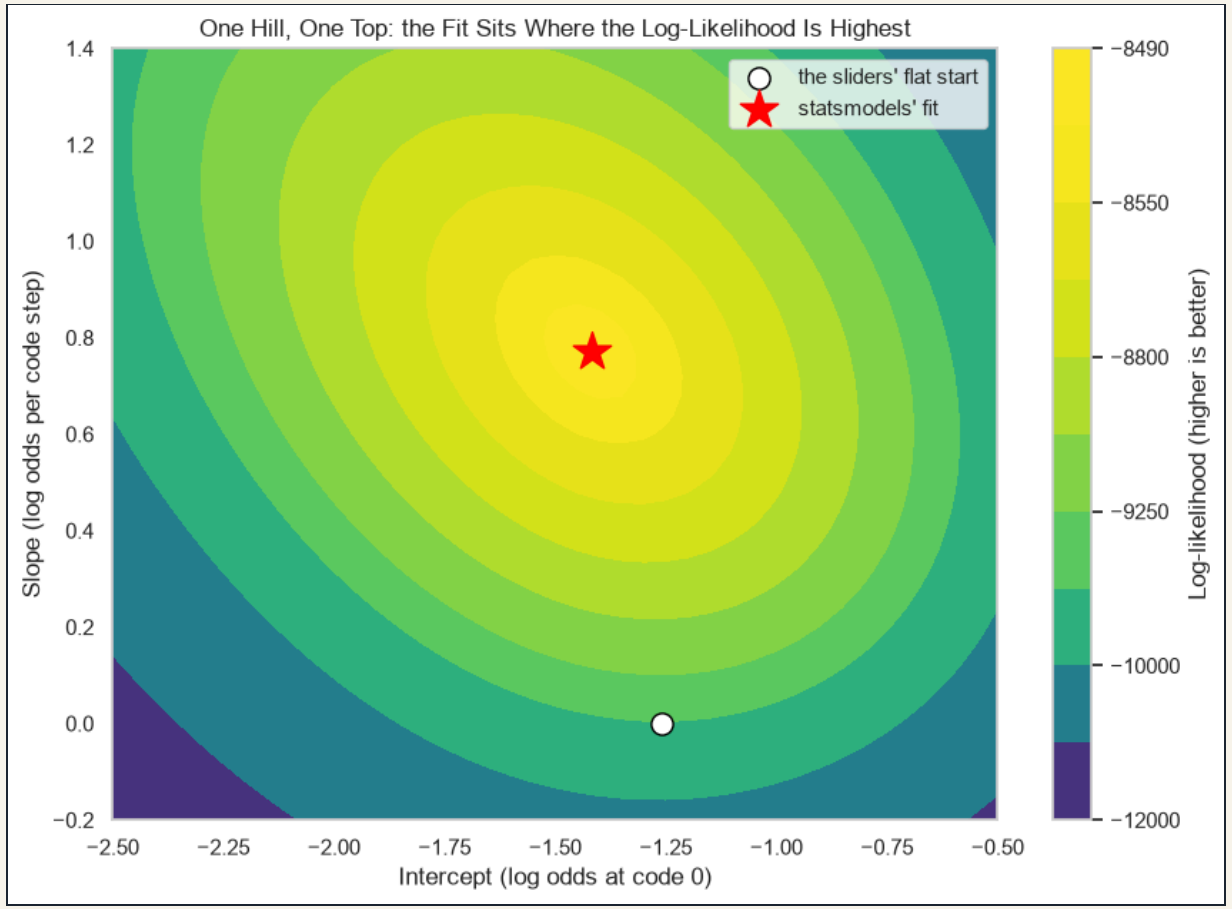


LOG LOSS BY MODEL



The 19-column pipeline is the week's model; Day 7 turns its probabilities into flags and counts what they catch.

Insights and Conclusion



THE GRID BEHIND THE HILL

```
intercepts = np.linspace(-2.5, -0.5, 41)
slopes = np.linspace(-0.2, 1.4, 41)
```

THE S CURVE AS CODE

```
def logistic(z):
    return 1 / (1 + np.exp(-z))
```

A straight line leaves 0 to 1; a logistic regression cannot, because its line lives in log odds and the S curve bends it back.

You Can Now

- Say why a straight line fails on a 0/1 outcome.
- Move a number between probability, odds and log odds.
- Fit a logistic regression with statsmodels and scikit-learn.
- Read a coefficient as an odds ratio, with its unit.
- Score probabilities with log loss on validation accounts.

NEXT
WEEK 2 · DAY 2

Confusion Matrix, Precision, and Recall

Day 7 turns the model's probabilities into flags and counts what they catch.